Organiser le travail en quatre niveaux : Epic, Feature, Story, Tâche

Note méthodologique — septembre 2026

---

1. Objet et contexte

Ce document décrit une organisation du backlog (liste ordonnée du travail à faire) en quatre niveaux : Epic, Feature, Story, Tâche. Il précise le rôle de chaque niveau, qui le rédige, qui le valide et comment l'écrire, à partir d'un exemple tiré d'un logiciel de simulation.

Les rôles et le vocabulaire nécessaires à la lecture sont définis en section 2. Un glossaire complet figure en section 12.

Origine de la hiérarchie. Le Scrum Guide (traductions disponibles ici), qui définit Scrum, ne parle ni d'epics ni de user stories : il ne connaît que des « éléments de backlog » qui doivent pouvoir être terminés en un sprint. La hiérarchie décrite ici vient de pratiques complémentaires, formalisées par le framework SAFe (Scaled Agile Framework) et intégrées nativement dans les outils : Azure DevOps (Epic → Feature → User Story → Task) et Jira (Epic → Story → Sub-task, extensible au-dessus). Les principes restent ceux de l'agilité : livrer souvent, par petits incréments vérifiables.

Point de départ : le backlog à plat. Une configuration Jira de base repose sur deux niveaux réellement utilisés : des stories et leurs sous-tâches. Le type Epic existe par défaut mais reste souvent peu exploité. Cette organisation suffit à faire tourner des sprints ; elle atteint ses limites dès que les demandes dépassent la taille d'un sprint :

- Des stories rattachées à aucune demande : impossible de savoir quand une demande métier est couverte (traçabilité manquante).
- Des stories découpées artificiellement pour tenir dans un sprint (« partie 1 », « partie 2 », « commencer X »), qui ne décrivent plus aucun comportement vérifiable.
- Une confusion entre la demande métier, qui s'étale sur plusieurs sprints, et le travail d'un sprint.
- Un suivi de sprint imprécis, faute de tâches courtes et assignées.

Apports par rôle

Acteur	Apport
Experts métier, dans leur rôle de Product Owner	Exprimer un besoin complet sans le découper eux-mêmes ; suivre l'avancement de chaque demande ; valider sur des comportements démontrés

Acteur	Apport
Scrum Master	Un backlog dont la structure se vérifie : chaque story a un parent, chaque feature a un plan de sprints
Équipes de développement et de modélisation	Des stories dimensionnées pour un sprint, découpées par ceux qui font le travail
Parties prenantes et encadrement	Un avancement lisible à deux échelles : par epic et feature sur plusieurs sprints, par tâche à l'intérieur du sprint

---

2. Rôles et vocabulaire

Les termes agiles désignent des **rôles**, pas des postes. Une même personne peut tenir un rôle en plus de sa fonction, et il n'est pas nécessaire de changer d'intitulé de poste pour adopter Scrum (Atlassian — Rôles et responsabilités Scrum). Dans un projet de simulation, les rôles se répartissent comme suit.

Experts métier — Product Owner (responsable produit)

- **Qui.** Les personnes qui connaissent le système réel (installations du navire, conduite, procédures) et l'usage attendu du simulateur. Ce sont elles qui savent ce que le logiciel doit faire et ce qu'un opérateur doit pouvoir y faire.
- **Rôle agile.** Elles tiennent le rôle de Product Owner. En Scrum, le Product Owner est la personne responsable de la valeur du produit et du contenu du backlog : ce qui y entre, ce qui n'y entre pas, dans quel ordre (Scrum Guide — Product Owner). SAFe le décrit comme la « voix du client » au sein de l'équipe (SAFe — Product Owner). Un expert métier qui décide de ce qui sera livré est donc, par définition, le Product Owner. La maîtrise de la méthode agile n'est pas un prérequis du rôle : c'est le Scrum Master qui l'apporte.
- **Deux règles du Scrum Guide.** Le Product Owner est une personne, pas un comité : si plusieurs experts interviennent, un seul tient l'arbitrage final sur un produit donné. Il peut déléguer le travail sur le backlog (rédaction, tenue de l'outil) tout en restant responsable des décisions.
- **Ce qu'ils font.** Rédigent les epics (en SAFe, ce travail est porté par l'Epic Owner (responsable d'epic), qui définit l'epic avec les parties prenantes et les experts du domaine) ; définissent les features avec l'équipe ; priorisent ; disent le « quoi » en Sprint Planning ; acceptent les stories ; valident les features en Sprint Review (revue de sprint) ; signalent les anomalies, créées au niveau story en type Bug puis priorisées.
- **Ce qu'ils ne font pas.** Ils ne rédigent ni stories ni tâches, et ne réalisent pas le travail du sprint : le découpage en éléments réalisables en un sprint demande une visibilité technique que seule l'équipe possède.

Scrum Master (facilitateur de la méthode)

- **Qui.** La personne qui gère le côté agile : organisation et animation des sprints, tenue de l'outil, appui au Product Owner. SAFe nomme ce rôle Scrum Master / Team Coach ; d'autres organisations parlent de coach agile ou de facilitateur.
- **Rôle.** Responsable de la méthode, pas du contenu. Le Scrum Guide le charge de faire tenir les événements (planning, daily, review, rétrospective) dans leur durée, de lever les obstacles, et de servir le Product Owner en l'aidant à trouver des techniques de gestion du backlog et à formuler des éléments clairs et concis (Scrum Guide — Scrum Master).
- **Dans la pratique.** Il peut tenir le backlog au quotidien (créer les tickets, vérifier les liens parent-enfant, préparer le refinement, suivre le burndown) par délégation du Product Owner. Les décisions de contenu et de priorité restent celles des experts métier.

Équipes de réalisation : développeurs et modélisateurs

- **Qui.** Les développeurs logiciel (back-end, interface homme-machine, intégration, déploiement) et les modélisateurs, ingénieurs qui construisent les modèles physiques des installations reproduites par la simulation (pompes, vannes, échangeurs, circuits).
- **Rôle.** En Scrum, tous sont « Developers » : le guide emploie ce mot pour simplifier et précise qu'il couvre chercheurs, analystes, scientifiques et autres spécialistes (Scrum Guide — Developers). Ils découpent les features en stories et les stories en tâches, dimensionnent, réalisent, testent et démontrent.

Utilisateurs finaux : les opérateurs

- Les personnes qui utiliseront le simulateur. Ce sont eux le « rôle » des user stories (« En tant qu'opérateur… »). Ils ne participent pas nécessairement au projet : les experts métier parlent en leur nom.

Parties prenantes (stakeholders) et encadrement

- **Qui.** Direction de programme, encadrement des équipes, équipes voisines : toute personne qui a besoin de connaître l'avancement sans participer au travail du sprint.
- **Rôle.** Assistent à la Sprint Review, où l'équipe présente ses résultats aux parties prenantes clés ; lisent l'avancement au niveau epic et feature sur plusieurs sprints, au niveau tâche dans le sprint ; passent par le Product Owner pour toute demande, le Scrum Guide précisant que quiconque veut modifier le backlog doit convaincre le Product Owner ; laissent aux équipes le découpage en stories et tâches, le guide réservant aux développeurs la manière de transformer un élément de backlog en incrément (Scrum Guide — Sprint Planning).

Qui décide quoi

Décision	Experts métier / Product Owner	Scrum Master	Équipes dev et modélisation	Parties prenantes
Contenu et priorité du backlog	Décident	Aide sur la méthode	Proposent	Influencent via le Product Owner
Découpage des features en stories	Valident	Facilite	Décident	—
Découpage des stories en tâches, estimation	—	Facilite	Décident	—
Acceptation d'une story	Décident	—	Démontrent	—
Validation d'une feature	Décident, en Sprint Review	Organise	Démontrent	Assistent
Tenue des événements du sprint	Participent	Anime	Participent	Assistent à la revue

Termes indispensables

Terme (EN)	Traduction	Sens dans ce document
Backlog	Carnet de travail	Liste ordonnée de tout ce qu'il reste à faire sur le produit
Sprint	Sprint (itération)	Période fixe, souvent deux semaines, à l'issue de laquelle des stories sont terminées et démontrées
Acceptance criteria	Critères d'acceptation	Conditions vérifiables qui disent quand un élément est réussi
Definition of Done	Définition de terminé	Exigences de qualité communes à tout élément livré : testé, relu, déployé, démontré
Refinement	Affinage du backlog	Activité continue de découpage et de précision des éléments à venir
Sprint Review	Revue de sprint	Session de fin de sprint où l'équipe démontre le résultat et où le métier valide

Les autres termes sont définis au fil du document et repris dans le glossaire (section 12).

---

3. Les quatre niveaux en un coup d'œil

```
EPIC          Demande métier complète (plusieurs mois)
└─ FEATURE    Capacité visible et testable par le métier (quelques
sprints)
   ├─ STORY   Comportement livrable en un sprint (par ex. 2 semaines)
   │  └─ TÂCHE   Unité de travail d'une personne (≤ 1 jour)
   └─ STORY
      └─ TÂCHE
```

Niveau	Rôle	Rédigé par	Horizon	Validé par
Epic (épopée)	Le besoin métier et ses critères de succès	Experts métier (Product Owner)	Plusieurs mois	Experts métier
Feature (fonctionnalité)	Une capacité que le métier peut voir et tester	Experts métier (Product Owner), avec l'équipe	Quelques sprints	Experts métier, en Sprint Review
Story (user story ou enabler story)	Un comportement livrable en un sprint	Équipes de développement et de modélisation	≤ 1 sprint	Product Owner
Tâche (sub-task, sous-tâche)	Une unité de travail d'une personne	La personne qui la réalise	≤ 1 jour	La personne qui la réalise

Règle commune : la tranche verticale. Une feature ou une story décrit un comportement observable (le modèle réagit, la vue l'affiche), jamais une couche technique isolée (« le back », « le front », « les tests »). Cette notion de *vertical slice (tranche verticale)* est expliquée dans le guide Humanizing Work.

Exemple fil rouge. Les sections suivantes déroulent un exemple volontairement simplifié : le sea water cooling circuit (circuit de refroidissement à l'eau de mer) d'un navire, avec ses pompes, ses vannes et son échangeur.

---

4. Six principes

1. **Les experts métier, dans leur rôle de Product Owner, écrivent les epics** avec des critères de succès et priorisent ; **l'équipe écrit les stories et les tâches** ; **le Scrum Master fait vivre la méthode**.
2. **Tout ticket a un parent** : chaque story appartient à une feature, chaque feature à une epic.
3. **Une story tient dans un sprint** et décrit un comportement vérifiable. Trop grosse ? Elle est découpée par comportement, jamais étalée sur deux sprints.

4. **Chaque feature a un plan de sprints** : dès son affinage, ses stories sont placées sur les sprints à venir, avec le sprint où chaque critère d'acceptation sera vérifié (section 8).
5. **Une tâche dure au plus une journée** et se suit en heures.
6. **Le métier valide en Sprint Review**, feature par feature, sur des comportements démontrés.

---

5. Niveau 1 : l'Epic

Définition. Une epic est une demande métier complète, trop grosse pour un sprint, souvent étalée sur plusieurs mois. Elle correspond à une demande métier ou à un regroupement cohérent de demandes. Voir la définition d'Atlassian.

Comment l'écrire. Le modèle ci-dessous reprend l'*epic hypothesis statement (énoncé d'hypothèse d'epic)* de SAFe : le besoin est formulé comme une hypothèse de valeur, accompagnée de résultats mesurables. Le modèle complet figure dans le gabarit de lean business case publié par Scaled Agile et Atlassian.

Sans critères de succès, personne ne peut dire quand l'epic est terminée. « La propulsion fonctionne » n'est pas une epic exploitable ; « l'opérateur peut jouer le scénario démarrage → régime établi → arrêt, validé par l'expert » en est une.

Modèle

Champ	Contenu attendu
Titre	Verbe + résultat métier
Pour	Les utilisateurs concernés
qui	Leur besoin
la solution	Le nom du module ou de la fonction
est un(e)	Ce que c'est, en une ligne
qui apporte	La valeur apportée
contrairement à	La situation actuelle
notre solution	Ce qui change
Résultats métier (business outcomes)	Bénéfices mesurables
Indicateurs avancés (leading indicators)	Mesures intermédiaires de progression
Exigences non fonctionnelles (NFR, non-functional requirements)	Performance, déterminisme, ergonomie…
Périmètre inclus / exclu	Ce qui est dedans, ce qui n'y est pas

Champ	Contenu attendu
Critères de clôture	Ce qui doit être vrai pour fermer l'epic

Exemple

Champ	Exemple
Titre	Conduire le circuit de refroidissement eau de mer en simulation
Pour	les opérateurs utilisant le simulateur
qui	doivent s'entraîner à la conduite du circuit
la solution	module « circuit de refroidissement »
est un	modèle physique + vues de conduite interactives
qui apporte	la manœuvre des pompes et des vannes, l'observation des températures et des alarmes
contrairement à	l'absence de ce circuit dans la simulation
notre solution	reproduit son comportement dynamique
Résultats métier	Scénario complet (démarrage → régime établi → arrêt) jouable sans intervention technique ; comportements validés par l'expert du circuit
Indicateurs avancés	Nombre de features validées en revue par le métier ; nombre d'anomalies ouvertes par feature
NFR	Rafraîchissement des vues fluide (cible fixée par le métier) ; simulation déterministe (scénario rejouable à l'identique)
Inclus	Pompes, vannes, échangeur, températures, alarmes
Exclu	Circuits connexes, pannes multiples simultanées
Critères de clôture	Scénario complet validé par l'expert métier

---

6. Niveau 2 : la Feature

Définition. Une feature est une capacité du système qui apporte de la valeur à une partie prenante et se livre en quelques sprints. Elle est définie par les experts métier, dans leur rôle de Product Owner, avec l'équipe, puis découpée en stories par les équipes. Voir Features and Capabilities (SAFe) et Agile process in Azure Boards (Microsoft).

Règle. Une feature est une tranche verticale : modèle et vue ensemble. Une feature « modélisation du circuit » et une feature « vues du circuit » séparées ne sont validables ni l'une ni l'autre par le métier. Le travail de modélisation vit dans la feature sous forme d'enabler stories (voir section 7).

Ses critères d'acceptation servent deux fois. D'abord à la validation finale en Sprint Review ; ensuite, dès l'affinage, à construire le plan de la feature : pour chaque critère, quelles stories le réalisent et dans quel sprint il sera vérifié (section 8).

Modèle

Champ	Contenu attendu
Titre	La capacité, en une phrase courte
Hypothèse de bénéfice (benefit hypothesis)	Ce que le métier gagne, formulé de façon vérifiable
Critères d'acceptation (acceptance criteria)	Comportements vérifiables en Sprint Review

Les features de l'epic

Feature	Hypothèse de bénéfice
F1 — Circulation pumps (pompes de circulation)	L'opérateur démarre et arrête les pompes et voit le débit
F2 — Hull and isolation valves (vannes de coque et d'isolement)	L'opérateur isole ou ouvre le circuit
F3 — Heat exchanger and temperatures (échangeur et températures)	L'opérateur voit l'effet du refroidissement
F4 — Alarms and failure modes (alarmes et modes de défaillance)	L'opérateur réagit à une panne de pompe ou de vanne

Exemple détaillé : F1

Champ	Exemple
Titre	Conduire les pompes de circulation
Hypothèse de bénéfice	L'opérateur met le circuit en circulation et vérifie le débit, sans intervention technique
Critères d'acceptation	Démarrage et arrêt de chaque pompe depuis la vue ; état marche/arrêt affiché ; débit animé cohérent avec l'état des pompes ; comportement validé par l'expert métier en Sprint Review

---

7. Niveau 3 : les Stories

Définition. Une story est un comportement livrable et vérifiable en un sprint. Elle est rédigée par l'équipe qui la réalise, sous la responsabilité du Product Owner. SAFe en distingue deux types :

- **User story (récit utilisateur)** : un comportement visible par l'utilisateur. C'est ce que le métier voit en Sprint Review. Voir aussi User stories (Atlassian).
- **Enabler story (story technique habilitante)** : le travail d'exploration, d'architecture, d'infrastructure ou de conformité (compliance) qui rend les user stories possibles ou en prouve le résultat. Dans un logiciel de simulation : le modèle physique, les flux de données, les études, la validation de bout en bout d'une feature.

Les deux types sont au même niveau et rattachés à la même feature. Si une feature ne contient que des enabler stories, le métier ne voit rien avant la fin : les deux types doivent alterner.

Règles

- **Taille** : la story tient dans un sprint. Le Scrum Guide ne considère « prêt » qu'un élément qui peut être terminé en un sprint. Repère du guide Humanizing Work : 6 à 10 stories par sprint.
- **Qualité** : les critères INVEST — Independent, Negotiable, Valuable, Estimable, Small, Testable (indépendante, négociable, porteuse de valeur, estimable, petite, testable).
- **Critères d'acceptation** : format Given / When / Then (étant donné / quand / alors), issu du BDD (behavior-driven development). Chaque critère décrit un comportement observable, pas une étape de réalisation. Écrits ainsi, les critères peuvent devenir des tests automatisés et documenter le comportement réel du système (Cucumber — spécifications exécutables).
- **Découpage** : par comportement (le cas simple d'abord, puis les variantes), jamais par couche technique. Deux méthodes de référence : le flowchart Humanizing Work, traduit en français, et SPIDR de Mike Cohn.

Ce qui n'est pas une story

Formulation	Problème
« Commencer la vue X » / « Finir la vue X »	Aucun critère testable : impossible de dire si c'est terminé
« Vue X — partie 1 » / « partie 2 »	Découpage par le calendrier, pas par le comportement
« Story front » / « Story back » / « Story tests »	Découpage horizontal : aucune des trois n'a de valeur seule
« En tant que développeur, je veux une base de données… »	Une tâche déguisée en story

Ces antipatterns sont décrits par Mountain Goat Software et Humanizing Work.

Modèle : user story

Champ	Contenu attendu
Titre	Le comportement, en quelques mots
Énoncé	En tant que [rôle], je veux [action] afin de [valeur]
Critères d'acceptation	Étant donné [contexte] — Quand [événement] — Alors [résultat observable] ; un bloc par cas

Modèle : enabler story

Champ	Contenu attendu
Titre	[Enabler] + le travail technique
Énoncé	Ce qui doit être fait, et pour quelle feature
Critères d'acceptation	Résultats techniques vérifiables : tests, données publiées, mesures

Les stories de F1

ID	Type	Titre	Sprint
EN-1	Enabler	Modèle de la pompe de circulation	N
EN-2	Enabler	Exposer l'état des pompes au frontal	N
US-1	User story	Afficher l'état marche/arrêt des pompes	N+1
US-2	User story	Démarrer et arrêter une pompe depuis la vue	N+1
US-3	User story	Afficher le débit animé (flow animation)	N+2
VAL-1	Enabler (validation)	Valider le scénario complet de F1 avec l'expert métier	N+2

La feature s'étale sur trois sprints ; aucune story ne déborde de son sprint. La répartition par sprint est expliquée en section 8.

Exemple EN-1

- **Titre** : [Enabler] Modèle de la pompe de circulation
- **Énoncé** : modéliser le comportement d'une pompe (marche/arrêt, débit nominal, montée en débit) pour la feature F1.
- **Critères d'acceptation** :

- commande « marche » → le débit atteint la valeur nominale en un temps de montée paramétrable ;
- commande « arrêt » → le débit redescend à zéro ;
- état et débit publiés dans le flux de sortie du modèle ;
- tests unitaires du modèle au vert.

Exemple US-1

- **Titre** : Afficher l'état marche/arrêt des pompes
- **Énoncé** : en tant qu'opérateur, je veux voir l'état de chaque pompe sur la vue circuit afin de savoir si le circuit est en circulation.
- **Critères d'acceptation** :
  - Étant donné la vue circuit ouverte, quand la pompe P1 passe en marche dans le modèle, alors le symbole P1 s'affiche en état « marche ».
  - Étant donné la vue circuit ouverte, quand la pompe P1 s'arrête, alors le symbole P1 s'affiche en état « arrêt ».
  - Étant donné la perte du flux de données, quand la vue ne reçoit plus d'état, alors les pompes s'affichent en état « inconnu ».

Exemple VAL-1

- **Titre** : [Enabler] Validation de la feature F1 — scénario complet
- **Énoncé** : rejouer le scénario « circuit à l'arrêt → démarrage P1 → démarrage P2 → arrêt P1 → arrêt général » sur la vue reliée au modèle, et comparer les débits affichés aux valeurs de référence du modèle.
- **Critères d'acceptation** :
  - le scénario est rejouable à l'identique (déterminisme) ;
  - à chaque étape, l'état et le débit affichés correspondent aux sorties du modèle ;
  - aucune anomalie bloquante ouverte sur F1 ;
  - validation prononcée par l'expert métier en Sprint Review.

Cas particulier : le sujet est inconnu. Quand personne ne sait comment implémenter, un spike (étude exploratoire) est créé : une enabler story limitée dans le temps, dont les critères d'acceptation sont les questions à résoudre. Les stories d'implémentation sont planifiées au sprint suivant. C'est le dernier des neuf patterns de découpage du guide Humanizing Work : à utiliser quand les autres ne s'appliquent pas.

---

8. De la feature au plan de sprints

Principe. Une feature dure plusieurs sprints ; ses stories, non. Dès que la feature est affinée, ses critères d'acceptation sont connus : l'équipe peut alors la dérouler sur les sprints à venir — quelles stories dans quel sprint, et, pour chaque critère, quand et comment il sera

vérifié. Ce **plan de feature** est établi en refinement, avec les experts métier et le Scrum Master, puis ajusté à chaque Sprint Planning.

C'est la logique du PI Planning en SAFe : les équipes découpent les features en stories et les placent dans les itérations à venir, en construisant d'abord un plan de bout en bout plutôt qu'itération par itération — planifier un sprint à la fois fait perdre la vue d'ensemble et les dépendances (Scaled Agile — PI Planning: plan to discover). Le backlog arrive ainsi en Sprint Planning déjà partiellement planifié ; le Product Owner y définit le « quoi », l'équipe le « comment » et le « combien » (SAFe — Iteration Planning).

Ordre naturel d'une feature de simulation

Sprint	Contenu	Ce qui est vérifié à la fin du sprint
N	Enabler stories : modèle physique, flux de données	Comportement du modèle par tests unitaires et d'intégration ; état et débit publiés
N+1	User stories : affichage, commandes	Chaque comportement d'interface par ses critères Given / When / Then ; démonstration au Product Owner
N+2	Dernières user stories + story de validation de la feature	Scénario de bout en bout, cohérence modèle ↔ vue, comparaison aux valeurs de référence ; validation par l'expert métier en Sprint Review

Deux niveaux de vérification, à ne pas confondre

Niveau	Quoi	Quand	Qui
Story	Les tests de la story : unitaires, intégration, critères d'acceptation	Dans le sprint de la story. Développement et test d'une même story ne se séparent jamais sur deux sprints : c'est la Definition of Done	L'équipe réalise et démontre ; le Product Owner accepte
Feature	L'intégration de toutes les stories, le scénario complet, la validation métier	Dans le dernier sprint de la feature, dans une story dédiée (VAL-1)	L'équipe démontre ; l'expert métier valide

La story de validation de feature est une enabler story : SAFe range le travail de vérification et de conformité parmi les enablers (SAFe — Story). Elle n'est pas la « story tests » de la section 7 : elle ne teste pas une story à la place de celle-ci, elle vérifie que les stories réunies produisent la capacité promise. En SAFe, cette démonstration intégrée des features, mesure objective de l'avancement, s'appelle le System Demo.

Plan de la feature F1

Critère d'acceptation de F1	Réalisé par	Vérifié par	Sprint de vérification
Démarrage et arrêt de chaque pompe depuis la vue	EN-1, EN-2, US-2	Critères de US-2 ; scénario VAL-1	N+1, puis N+2
État marche/arrêt affiché	EN-2, US-1	Critères de US-1 (trois états)	N+1
Débit animé cohérent avec l'état des pompes	EN-1, US-3	Critères de US-3 ; comparaison au débit du modèle dans VAL-1	N+2
Comportement validé par l'expert métier	VAL-1	Sprint Review du sprint N+2	N+2

Ce tableau se lit dans les deux sens : chaque critère sait par quelles stories il sera réalisé et quand il sera prouvé ; chaque story sait quel critère de la feature elle sert. Une story qui ne sert aucun critère n'a pas sa place dans la feature.

---

9. Niveau 4 : les Tâches

Définition. Une tâche est une unité de travail d'une personne, d'une journée au plus. Elle ne livre pas de valeur en elle-même : elle sert à suivre l'avancement de la story au quotidien. C'est le niveau où se lit, dans le sprint, où en est chaque développeur ou modélisateur.

Règles

- **Durée ≤ 1 jour** : le Scrum Guide décrit le découpage des éléments sélectionnés en unités de travail d'une journée ou moins.
- **Estimation en heures sur la tâche, story points sur la story.** Jira décrit cette séparation estimation / suivi comme une pratique courante : Configuring estimation and tracking.
- **Un responsable, un résultat vérifiable.**
- **Le reste à faire (remaining estimate) est mis à jour chaque jour** : c'est ce qui alimente le burndown (courbe d'avancement) du sprint et les barres de capacité par personne. Voir Sprints and Scrum key concepts (Microsoft).

Modèle

Champ	Contenu attendu
Titre	Verbe à l'infinitif + objet précis + résultat vérifiable
Estimation	Heures

Champ	Contenu attendu
Responsable	Une personne

Trop vague	Exploitable
Vue pompes	Créer le composant Angular `PumpComponent` avec entrée d'état typée
Tests	Écrire les tests du composant pour les états marche / arrêt / inconnu
Modèle	Implémenter la montée en débit avec temps de montée paramétrable

Tâches de EN-1 (équipe modélisation)

Tâche	Estimation
Implémenter la commande marche/arrêt et le débit nominal	6 h
Implémenter la montée et la descente en débit (temps paramétrable)	5 h
Publier l'état et le débit dans le flux de sortie du modèle	4 h
Écrire les tests unitaires (démarrage, arrêt, montée en débit)	4 h
Faire relire le code et corriger	2 h

Tâches de US-1 (équipe développement)

Tâche	Estimation
Dessiner le symbole SVG pompe (états marche / arrêt / inconnu)	4 h
Créer le composant Angular `PumpComponent` avec entrée d'état typée	5 h
Brancher le composant sur le flux d'état des pompes	4 h
Placer P1 et P2 sur la vue circuit	2 h
Écrire les tests du composant (trois états)	3 h
Déployer sur l'environnement de test et vérifier	2 h
Démontrer au Product Owner et vérifier les critères d'acceptation	1 h

Tâches récurrentes. Tests, relecture, déploiement et démonstration reviennent dans chaque story : elles découlent de la Definition of Done. Les prévoir dans un modèle de story évite de les oublier.

---

10. Déroulement d'un sprint

1. **Refinement (affinage)**, en continu, animé par le Scrum Master : les experts métier et les équipes découpent les prochaines features en stories dimensionnées pour un sprint et établissent ou mettent à jour le plan de chaque feature (section 8). Un élément non affiné n'entre pas en Sprint Planning (Scrum.org).
2. **Sprint Planning** : à partir des plans de feature, le Product Owner dit le « quoi » ; l'équipe choisit les stories du sprint, fixe un Sprint Goal (objectif de sprint) et découpe les stories en tâches.
3. **Daily Scrum (mêlée quotidienne)** : chacun met à jour ses tâches et son reste à faire.
4. **Sprint Review** : l'équipe démontre les stories terminées aux experts métier et aux parties prenantes ; une feature est validée quand tous ses critères d'acceptation sont vérifiés, en général lors de la revue du sprint qui contient sa story de validation.
5. **Retrospective (rétrospective)** : l'équipe ajuste sa façon de travailler.

Une story n'est pas terminée en fin de sprint. Elle retourne au backlog, elle est ré-estimée et la cause est traitée en rétrospective. Le Scrum Guide précise qu'un élément qui ne respecte pas la Definition of Done ne peut pas être présenté en Sprint Review. Elle n'est pas renommée « partie 2 » ; à la clôture du sprint, Jira propose de la renvoyer au backlog ou vers un sprint futur (Complete a sprint). Le plan de la feature est alors décalé et le Product Owner en est informé.

Une feature ne mobilise pas toutes les spécialités dans le même sprint. Le séquencement entre spécialités se lit au niveau feature, sur plusieurs sprints : modèle au sprint N, vue au sprint N+1, validation au sprint N+2. Dans un même sprint, l'équipe modélisation peut avancer sur les enabler stories de F2 pendant que l'équipe développement livre les user stories de F1.

---

11. Représentation dans Jira

Niveau de la méthode	Type Jira	Remarque
Epic métier	Niveau au-dessus d'Epic, ou label commun	Voir contraintes ci-dessous
Feature	Epic Jira	Le rapport Epic Burndown suit l'avancement de la feature ; le plan de feature se lit dans le champ Sprint de ses stories
User story / Enabler story	Story / type « Enabler » à créer	Même niveau hiérarchique
Tâche	Sub-task	Estimation initiale et reste à faire en heures

Contraintes de l'outil

- Jira ne propose nativement que trois niveaux : Epic → Story → Sub-task. Un niveau supplémentaire au-dessus d'Epic demande Jira Premium ou Enterprise en Cloud, ou Advanced Roadmaps, inclus dans Data Center. Il est impossible d'insérer un niveau entre Epic et Story ; la pratique courante consiste à renommer Epic en « Feature » et à créer un niveau « Epic » au-dessus (Atlassian Community).
- Une sous-tâche hérite toujours du sprint de sa story : un travail qui doit s'étaler sur plusieurs sprints se traite en plusieurs stories sous une même feature, pas en sous-tâches réparties (JRACLOUD-98079).
- Contrôle des stories orphelines : `issuetype = Story AND "Epic Link" is EMPTY` (Data Center) ou `issuetype = Story AND parent is EMPTY` (Cloud). Cette vérification relève du Scrum Master.
- Suivi d'une feature : le rapport Epic Burndown montre le rythme d'avancement et prévoit le nombre de sprints restants.

---

12. Glossaire

Rôles

Terme (EN)	Traduction	Définition courte
Expert métier	—	Personne qui connaît le système réel et l'usage attendu du simulateur ; tient le rôle de Product Owner
Product Owner	Responsable produit	Rôle responsable du contenu et de l'ordre du backlog, et de la valeur livrée ; une personne, pas un comité
Scrum Master	Facilitateur de la méthode	Rôle responsable de la méthode : anime les événements, tient l'outil, aide le Product Owner, lève les obstacles ; ne décide ni du contenu ni des priorités
Developers	Équipe de réalisation	Toute personne qui construit l'incrément : développeurs logiciel et modélisateurs
Modélisateur	—	Ingénieur qui construit les modèles physiques des installations simulées
Utilisateur final	Opérateur	Personne qui utilisera le simulateur ; le « rôle » des user stories
Stakeholders	Parties prenantes	Direction de programme, encadrement, équipes voisines : suivent l'avancement, assistent à la Sprint Review, passent par le Product Owner pour toute demande
Epic Owner	Responsable d'epic	En SAFe, personne qui définit une epic avec les parties prenantes et les experts du domaine ; ici, l'expert métier

Niveaux du backlog

Terme (EN)	Traduction	Définition courte
Backlog	Carnet de travail	Liste ordonnée de tout ce qu'il reste à faire sur le produit
Epic	Épopée	Demande métier complète, sur plusieurs mois, découpée en features
Feature	Fonctionnalité	Capacité visible et testable par le métier, sur quelques sprints
User story	Récit utilisateur	Comportement visible par l'utilisateur, livrable en un sprint
Enabler story	Story technique habilitante	Travail technique (modèle, architecture, étude, validation) qui rend les user stories possibles ou en prouve le résultat
Spike	Étude exploratoire	Enabler story limitée dans le temps pour lever une incertitude
Task / Sub-task	Tâche / sous-tâche	Unité de travail d'une personne, une journée au plus
Plan de feature	—	Répartition des stories d'une feature sur les sprints à venir, avec le sprint de vérification de chaque critère
Bug	Anomalie	Dysfonctionnement constaté, créé au niveau story et priorisé par le Product Owner

Événements et pratiques

Terme (EN)	Traduction	Définition courte
Sprint	Sprint (itération)	Période fixe, souvent deux semaines, à l'issue de laquelle des stories sont terminées
Sprint Goal	Objectif de sprint	Objectif unique du sprint, qui donne sa cohérence au travail
Refinement	Affinage du backlog	Activité continue de découpage et de précision des éléments à venir
Sprint Planning	Planification de sprint	Réunion de début de sprint : choix des stories, découpage en tâches
Daily Scrum	Mêlée quotidienne	Point quotidien de 15 minutes des développeurs
Sprint Review	Revue de sprint	Session de fin de sprint où l'équipe présente le résultat aux parties prenantes et où le métier valide

Terme (EN)	Traduction	Définition courte
System Demo	Démonstration système	En SAFe, démonstration intégrée des features de toutes les équipes, mesure objective de l'avancement
Retrospective	Rétrospective	Réunion de fin de sprint sur la façon de travailler
PI Planning	Planification d'incrément	En SAFe, événement où les équipes placent les stories des features sur les itérations à venir

Qualité et mesure

Terme (EN)	Traduction	Définition courte
Acceptance criteria	Critères d'acceptation	Conditions vérifiables qui définissent quand un élément est réussi
Definition of Done	Définition de terminé	Exigences de qualité communes à tout élément livré
Given / When / Then	Étant donné / quand / alors	Format de critère d'acceptation décrivant contexte, événement et résultat observable ; peut devenir un test automatisé
Vertical slice	Tranche verticale	Comportement complet qui traverse toutes les couches (modèle, back, front)
INVEST	—	Six qualités d'une bonne story : indépendante, négociable, porteuse de valeur, estimable, petite, testable
Story points	Points d'effort	Unité relative de taille d'une story, servant à mesurer la vélocité
Velocity	Vélocité	Nombre de story points terminés par sprint, en moyenne
Burndown	Courbe d'avancement	Graphique du travail restant au fil du sprint ou de la feature

---

13. Sources

Référentiel et rôles

- Scrum Guide 2020 — https://scrumguides.org/scrum-guide.html (traductions : https://scrumguides.org/download.html)
- Atlassian — Rôles et responsabilités Scrum : https://www.atlassian.com/fr/agile/scrum/roles
- SAFe — Product Owner : https://framework.scaledagile.com/product-owner

- SAFe — Scrum Master / Team Coach : https://framework.scaledagile.com/scrum-master-team-coach/
- SAFe — Epic Owner : https://framework.scaledagile.com/epic-owner
- Scaled Agile — Who writes user stories : https://scaledagile.com/blog/who-writes-user-stories/

Hiérarchie Epic / Feature / Story

- SAFe — Features and Capabilities : https://framework.scaledagile.com/features-and-capabilities
- SAFe — Story (user stories et enabler stories) : https://framework.scaledagile.com/story
- SAFe — Spikes : https://framework.scaledagile.com/spikes
- Atlassian — Epics, stories et initiatives : https://www.atlassian.com/fr/agile/project-management/epics-stories-themes
- Atlassian — User stories : https://www.atlassian.com/agile/project-management/user-stories
- Atlassian / Scaled Agile — Gabarit lean business case (epic hypothesis statement) : https://www.atlassian.com/software/confluence/templates/safe-lean-business-case
- Microsoft Learn — Agile process in Azure Boards : https://learn.microsoft.com/en-us/azure/devops/boards/work-items/guidance/agile-process-workflow
- Microsoft Learn — Sprints and Scrum key concepts : https://learn.microsoft.com/en-us/azure/devops/boards/sprints/scrum-key-concepts

Planification et vérification

- Scaled Agile — PI Planning: plan to discover : https://scaledagile.com/blog/pi-planning-plan-to-discover/
- SAFe — Iteration Planning : https://www.scaledagileframework.com/iteration-planning
- SAFe — System Demo : https://framework.scaledagile.com/system-demo
- Cucumber — Spécifications exécutables (Given / When / Then) : https://cucumber.io/docs/guides/overview

Découpage des stories

- Humanizing Work — Guide to Splitting User Stories : https://www.humanizingwork.com/the-humanizing-work-guide-to-splitting-user-stories/
- Mountain Goat Software — SPIDR : https://www.mountaingoatsoftware.com/blog/five-simple-but-powerful-ways-to-split-user-stories
- Mountain Goat Software — Story splitting so teams can finish : https://www.mountaingoatsoftware.com/blog/story-splitting-how-to-split-user-stories-so-teams-can-finish
- Scrum.org — If your backlog is not refined : https://www.scrum.org/resources/blog/if-your-backlog-not-refined-then-you-are-doing-it-wrong

Outillage Jira

- Configuring estimation and tracking : https://confluence.atlassian.com/jirasoftwareserver/configuring-estimation-and-tracking-938845319.html
- Complete a sprint : https://support.atlassian.com/jira-software-cloud/docs/complete-a-sprint/
- Epic Burndown report : https://support.atlassian.com/jira-software-cloud/docs/view-and-understand-the-epic-burndown-report/
- Custom work type hierarchy (Cloud Premium) : https://confluence.atlassian.com/jirakb/create-a-custom-work-type-hierarchy-in-jira-cloud-1387599403.html
- Advanced Roadmaps inclus dans Data Center : https://www.atlassian.com/blog/jira/jira-data-center-includes-advanced-roadmaps
- Sous-tâches et sprints (JRACLOUD-98079) : https://jira.atlassian.com/browse/JRACLOUD-98079
- Atlassian Community — hiérarchie des types de tickets : https://community.atlassian.com/forums/Jira-questions/ISSUE-TYPE-HIERARCHY/qaq-p/3187114